```
      end
    end
  end
end
```

Alternatively, you can use up and down instead of change:

```
class ChangeProductsPrice < ActiveRecord::Migration
  def up
    change_table :products do |t|
      t.change :price, :string
    end
  end

  def down
    change_table :products do |t|
      t.change :price, :integer
    end
  end
end
```

2 Creating a Migration

2.1 Creating a Standalone Migration

Migrations are stored as files in the db/migrate directory, one for each migration class. The name of the file is of the form YYYYMMDDHHMMSS_create_products.rb, that is to say a UTC timestamp identifying the migration followed by an underscore followed by the name of the migration. The name of the migration class (CamelCased version) should match the latter part of the file name. For example 20080906120000_create_products.rb should define class CreateProducts and 20080906120001_add_details_to_products.rb should define AddDetailsToProducts. Rails uses this timestamp to determine which migration should be run and in what order, so if you're copying a migration from another application or generate a file yourself, be aware of its position in the order.

Of course, calculating timestamps is no fun, so Active Record provides a generator to handle making it for you:

```
$ bin/rails generate migration AddPartNumberToProducts
```

This will create an empty but appropriately named migration:

```
class AddPartNumberToProducts < ActiveRecord::Migration
  def change
  end
end
```

If the migration name is of the form "AddXXXToYYY" or "RemoveXXXFromYYY" and is followed by a list of column names and types then a migration containing the appropriate add_column and remove_column statements will be created.

```
$ bin/rails generate migration AddPartNumberToProducts
part_number:string
```

will generate

```
class AddPartNumberToProducts < ActiveRecord::Migration
  def change
    add_column :products, :part_number, :string
  end
end
```

If you'd like to add an index on the new column, you can do that as well: